

SKILL 16

```
GNU nano 2.7.1
#include <stdio.h>
#include <stdlib.h>
#include <unistd.h>
#include <sys/types.h>
#include <sys/wait.h>
#include <string.h>

#define MAX_JOBS 5

typedef struct {
    int job_id;
    pid_t pid;
    char command[100];
    char status[20];
} Job;

Job jobs[MAX_JOBS];
int job_count = 0;

void add_job(pid_t pid, const char *command) {
    if (job_count < MAX_JOBS) {
        jobs[job_count].job_id = job_count + 1;
        jobs[job_count].pid = pid;

        strcpy(jobs[job_count].command, command);
        strcpy(jobs[job_count].status, "Running");

        job_count++;
    }
}
```

```
void add_job(pid_t pid, const char *command) {
    if (job_count < MAX_JOBS) {
        jobs[job_count].job_id = job_count + 1;
        jobs[job_count].pid = pid;

        strcpy(jobs[job_count].command, command);
        strcpy(jobs[job_count].status, "Running");

        job_count++;
    }
}

void display_jobs() {
    printf("\n=== Job Table ===\n");

    for (int i = 0; i < job_count; i++) {
        printf("Job ID: %d | PID: %d | Command: %s | Status: %s\n",
            jobs[i].job_id,
            jobs[i].pid,
            jobs[i].command,
            jobs[i].status);
    }
}
```

```

void update_jobs() {
    int status;

    for (int i = 0; i < job_count; i++) {
        pid_t result = waitpid(jobs[i].pid, &status, WNOHANG);

        if (result > 0) {
            strcpy(jobs[i].status, "Completed");
        }
    }
}

void remove_completed_jobs() {
    int j = 0;

    for (int i = 0; i < job_count; i++) {
        if (strcmp(jobs[i].status, "Completed") != 0) {
            jobs[j] = jobs[i];
            j++;
        }
    }

    job_count = j;
}

int main() {

```

```

int main() {
    printf("=== Task 16: Background Job Management ===\n");

    printf("\n=== Launch Background Job ===\n");

    pid_t pid = fork();

    if (pid < 0) {
        perror("fork");
        return 1;
    }

    if (pid == 0) {
        printf("Background child started, PID = %d\n", getpid());

        sleep(5);

        printf("Background child finished\n");

        exit(0);
    }

    else {
        add_job(pid, "sleep 5");

        printf("Background job started with PID = %d\n", pid);
        printf("Prompt returned immediately to parent process\n");
    }

    display_jobs();

    printf("\n=== Monitor Background Job ===\n");

    sleep(2);

    update_jobs();

    display_jobs();

    printf("\nWaiting for job to finish...\n");

    sleep(4);

    update_jobs();

    display_jobs();
}

```

```
printf("\n=== Remove Completed Jobs ===\n");

remove_completed_jobs();

display_jobs();

printf("\nCompleted jobs removed from job table\n");

printf("\n=== Job Record Validation ===\n");

if (job_count == 0) {
    printf("All completed jobs removed successfully\n");
} else {
    printf("Active jobs still present\n");
}

printf("\n=== Task 16 Complete ===\n");

return 0;
}
```

```
printf("\n=== Remove Completed Jobs ===\n");

remove_completed_jobs();

display_jobs();

printf("\nCompleted jobs removed from job table\n");

printf("\n=== Job Record Validation ===\n");

if (job_count == 0) {
    printf("All completed jobs removed successfully\n");
} else {
    printf("Active jobs still present\n");
}

printf("\n=== Task 16 Complete ===\n");

return 0;
}
```

Output:

```
=== Task 16: Background Job Management ===

=== Launch Background Job ===
Background job started with PID = 583
Prompt returned immediately to parent process

=== Job Table ===
Job ID: 1 | PID: 583 | Command: sleep 5 | Status: Running

Background child started, PID = 583
=== Monitor Background Job ===

=== Job Table ===
Job ID: 1 | PID: 583 | Command: sleep 5 | Status: Running

Waiting for job to finish...
Background child finished

=== Job Table ===
Job ID: 1 | PID: 583 | Command: sleep 5 | Status: Completed

=== Remove Completed Jobs ===

=== Job Table ===

Completed jobs removed from job table

=== Job Record Validation ===
All completed jobs removed successfully
```